

Курс: Углубленное машинное обучение и искусственный интеллект в Python

Практическое задание: Рекуррентные нейронные сети (RNN) и прогнозирование временных рядов

Цель работы: Изучить архитектуру рекуррентных нейронных сетей (SimpleRNN, LSTM, GRU), научиться подготавливать данные временных рядов с помощью скользящего окна и строить модели для прогнозирования числовых последовательностей в TensorFlow/Keras.

Шаг 1. Подготовка данных и создание скользящего окна

В качестве источника данных выберите любой временной ряд (например, стоимость акций, климатические показатели или объемы продаж). Загрузите его и подготовьте для RNN:

1. Проведите масштабирование данных (рекомендуется MinMaxScaler, так как RNN чувствительны к масштабу входных величин).
2. Реализуйте функцию генерации признаков методом **скользящего окна (sliding window)**. На основе прошлых значений длиной N (размер окна) модель должна предсказывать следующее значение временного ряда.
3. Разделите данные на обучающую (70%), валидационную (15%) и тестовую (15%) выборки **без перемешивания** (параметр `shuffle=False`), чтобы сохранить хронологический порядок.

```
import numpy as np
```

```
def create_dataset(series, window_size):  
    X, y = [], []  
    for i in range(len(series) - window_size):  
        X.append(series[i:(i + window_size)])  
        y.append(series[i + window_size])  
    return np.array(X), np.array(y)
```

```
# Преобразуйте ваши данные: форма X должна быть (samples,  
window_size, features)
```

Шаг 2. Построение и сравнение архитектур RNN

Вам предстоит спроектировать и обучить три независимые конфигурации нейронных сетей для решения одной и той же задачи прогнозирования. Каждая модель должна принимать тензор формы (window_size, 1).

Требуемые архитектуры для реализации:

1. **Модель 1 (SimpleRNN):** Базовая рекуррентная сеть из 1-2 слоев SimpleRNN.
2. **Модель 2 (LSTM):** Сеть на базе долгосрочной краткосрочной памяти из 1-2 слоев LSTM.
3. **Модель 3 (GRU):** Сеть на базе управляемых рекуррентных блоков из 1-2 слоев GRU.

Важное правило Keras: Если вы соединяете несколько рекуррентных слоев друг за другом, у всех слоев, кроме последнего, должен быть установлен флаг `return_sequences=True`. На последнем слое оставьте `return_sequences=False`, чтобы передать плоский вектор в финальный слой `Dense(1)`.

```
import tensorflow as tf

# Пример структуры для LSTM (настройте параметры и аналогично
# соберите SimpleRNN и GRU)
model_lstm = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(window_size, 1)),
    tf.keras.layers.LSTM(64, return_sequences=True),
    tf.keras.layers.LSTM(32, return_sequences=False),
    tf.keras.layers.Dense(1)
])
```

Шаг 3. Обучение моделей

- Скомпилируйте каждую из трех моделей с оптимизатором `adam` и функцией потерь `mse` (метрика — `mae`).
- Обучите их на подготовленных данных одинаковое количество эпох (например, 20–30 эпох) с использованием валидационной выборки для контроля переобучения.

Шаг 4. Анализ результатов и визуализация

1. Сделайте прогноз (inference) для тестовой выборки каждой из трех моделей.
2. Выполните **обратное преобразование масштаба (inverse_transform)** предсказанных значений, чтобы вернуть их к исходным физическим величинам.
3. Постройте совмещенный график исходного временного ряда (Ground Truth) и прогнозных кривых от SimpleRNN, LSTM и GRU на тестовом интервале.
4. Рассчитайте метрики **MSE, MAE и R2** для каждого типа архитектуры на тестовых данных.
5. Оформите сравнительную таблицу результатов и сделайте вывод: какая архитектура оказалась наиболее точной, как сильно отличаются затраты времени на обучение и подтвердилась ли теоретическая устойчивость LSTM/GRU к затуханию градиентов по сравнению со стандартной RNN на вашем временном ряду.